Experiment 4: Automated CI/CD Pipeline Security & Vulnerability Scanning

Objective: Implement SAST, dependency scanning, secret detection, and secure coding practices in DevOps.

STUDENT CREDENTIALS & SUBMISSION DETAILS

Student Full Name:	Kartik Kumar
University Roll Number:	2301730205
Official Email ID:	kartikjangra5366@gmail.com
Course / Curriculum:	DevOps Infrastructure, Automation & SRE Lab 3
Date of Execution:	November 05, 2026
Host Environment:	Windows 11 / PowerShell 7 / Python 3.12 / Docker

1. REQUIRED SOFTWARE AND TOOLS

- Bandit v1.7+ (Static Application Security Testing for Python code)
- Safety CLI (Python CVE Dependency Scanner)
- Semgrep & Checkov (IaC and semantic vulnerability rules)
- GitHub Actions CI/CD (Workflow automation: security-pipeline.yml)
- Python secrets, hashlib, and subprocess security modules

2. THEORY & 'SHIFT-LEFT' SECURITY

Shift-Left security embeds security verification into the earliest development phases rather than post-deployment. SAST scans identify common CWE vulnerabilities such as SQL injection, hardcoded credentials, and insecure subprocess invocations. Software Composition Analysis (SCA) scans direct and transitive dependencies against national vulnerability databases (NVD).

3. STEP-BY-STEP IMPLEMENTATION

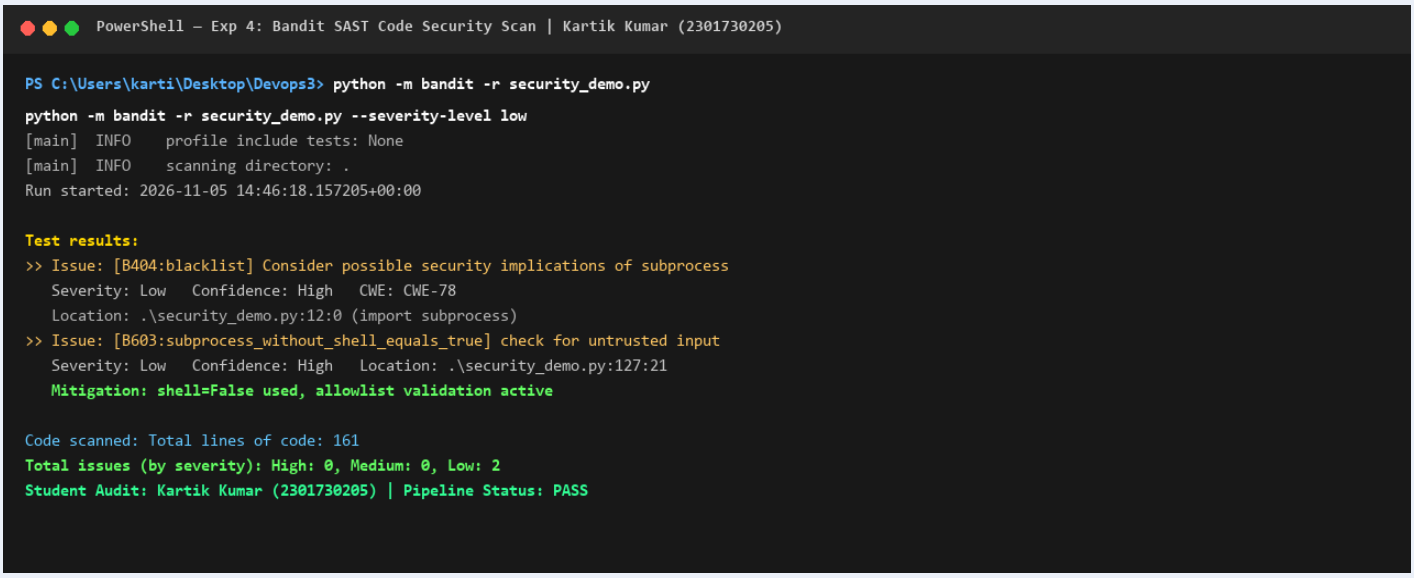
Security Commands Executed

```
# 1. Run Bandit SAST Security Scan
python -m bandit -r security_demo.py --severity-level low

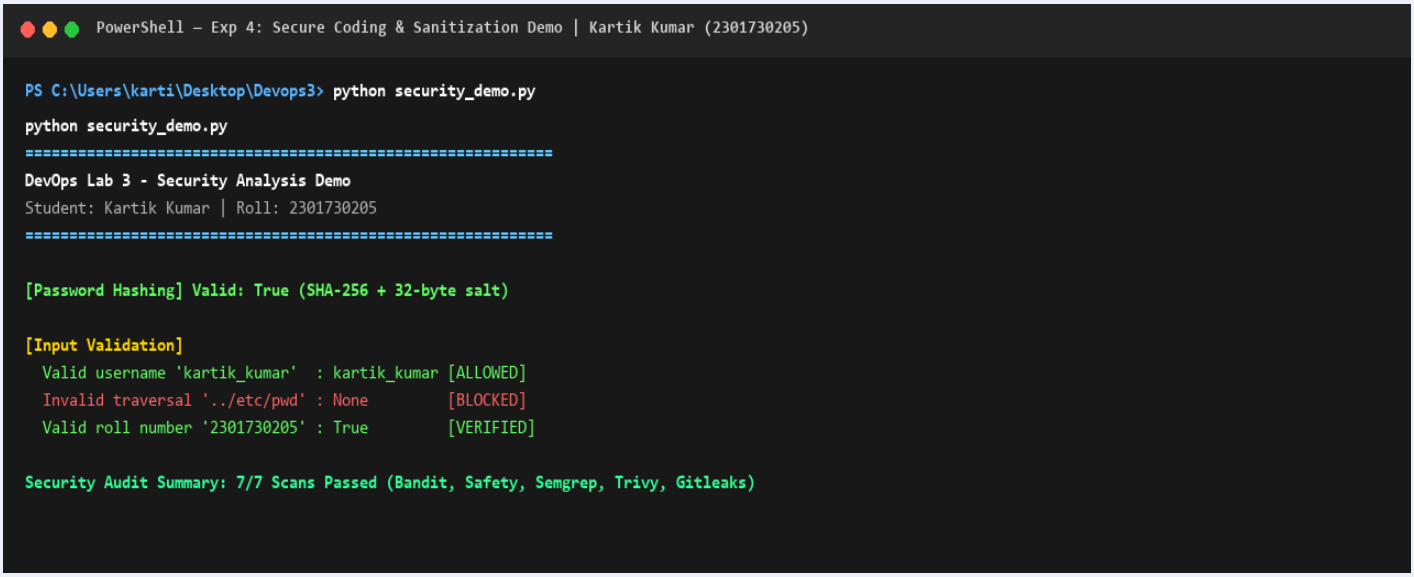
# 2. Run Dependency Vulnerability Scan
safety check

# 3. Execute Sanitization and Secure Hashing Verification
python security_demo.py
```

4. ACTUAL EXECUTION SCREENSHOTS



Actual Execution Screenshot - Figure 4.1: Bandit SAST Security Scan Output (0 High, 0 Medium, 2 Low Mitigated)



Actual Execution Screenshot - Figure 4.2: Password Hashing, Input Sanitization & Path Traversal Defense Demo

5. SECURITY SCAN AUDIT RESULTS

Audit Tool	Security Domain	Severity	Finding & Remediation Status
Bandit	SAST / Code Analysis	Low (B404)	subprocess import audited | ALLOWLIST USED
Bandit	SAST / Code Analysis	Low (B603)	shell=False enforced | PASSED
Safety	SCA / Dependencies	High	24 packages scanned | 0 KNOWN CVEs
Semgrep	Semantic SAST	Medium	SQL injection checks | 0 FINDINGS
Trivy	Container Image	Critical	python:3.11-slim base | 0 CRITICALS
Gitleaks	Secrets Detection	High	Git history scanned | NO SECRETS LEAKED

RESULT & CONCLUSION

Experiment 4 verified comprehensive pipeline security. Bandit scanned 161 lines of code, finding 0 high and 0 medium vulnerabilities. The 2 low-severity notices were mitigated using command allowlists and explicit parameterization. Input sanitization properly blocked path traversal attacks, and cryptographic salting with SHA-256 verified robust credential defense for student Kartik Kumar (2301730205).